chain by returning a response, or halt further processing. Error handling is included to prevent a single failing middleware from crashing the entire request.

Most importantly, this pattern makes it easy to add middleware that interfaces with external APIs or security services. For example, you could integrate with a threat intelligence platform to identify and block malicious requests or use a third-party service to perform advanced authentication or authorization checks.

9.2 Building a core security and data protection pipeline

Once we have some security elements in place, the next list of security considerations is related to strengthening the security model of the whole application. That includes how we allow users to interact with our application. If we are using external AI providers like OpenAI, we must implement fair and robust security controls to prevent overuse of our API credits. This means allowing very limited interactions with visitors (such as unauthenticated users) and then prompting them to authenticate so that users are enrolled with a quoted plan. This strategy helps you control costs, prevent abuse, and ensure fair access to your application's resources.

It's also crucial to manage API keys and secrets securely, considering all the potential threats. Anti-patterns like hardcoding API keys in the application code are one of those things that can hurt the overall security model.

Data management and compliance are of utmost importance if you are dealing with personally identifiable information (PII). Nowadays, many developers choose to offload authentication to dedicated providers like Google, but even if you are only storing information like emails or passwords, you must ensure they are handled with the highest levels of security. In short, to build a robust security and compliance pipeline for your AI web applications in production, pay attention to authentication/authorization, API key management, rate limiting, and data security (figure 9.2).

Let's start by discussing how to establish a secure authentication and authorization framework for your AI web application.

9.3 Setting up authentication and authorization

Given the computational resources and API costs associated with AI models, implementing robust authentication and authorization is crucial to controlling access and preventing abuse. We definitely don't want to allow every visitor on the site to abuse the AI interactions by spamming messages, and even if they do log in, we definitely want to impose quotas and reasonable limits. Remember that most commercial LLMs cost money and are quite expensive compared to traditional services if you don't carefully manage their usage.

How do we implement robust authentication and authorization controls? First, let's clarify what these are. *Authentication* is determining a user's identity. *Authorization* is determining what a user is allowed to do—in other words, what is this user allowed to access, or how many messages are they allowed to send to the AI web application?